

Raspberry Pi 3 Model B+ in Clustered Sensor Array Management

Hardware Overview of Raspberry Pi 3 Model B+

The Raspberry Pi 3 Model B+ is the final revision of the third-generation Pi, introduced in 2018 as an enhanced version of the 3B. It features improved processing speed, networking, and wireless capabilities while retaining the same 85×56 mm board size and 40-pin GPIO layout as the Pi 3 Model B ¹. Below is a summary of the key hardware specifications for the 3B+:

- **Processor (CPU):** Broadcom BCM2837B0 system-on-chip with a 64-bit quad-core ARM Cortex-A53 CPU, clocked at **1.4 GHz** (an increase from 1.2 GHz on the 3B) ² ³. This provides roughly a 15–17% boost in CPU performance over the original Pi 3 Model B ³. The BCM2837B0 revision includes an integrated metal heat-spreader, improving thermal dissipation and enabling the higher clock speed without rapid throttling ⁴.
- **Graphics (GPU):** Broadcom **VideoCore IV** GPU (part of the SoC), running at 400 MHz. It supports OpenGL ES 2.0 and hardware-accelerated video codecs (H.264 decode and encode at 1080p30) ⁵. This GPU can handle Full HD display output via HDMI and camera streaming/encoding tasks at 1080p, useful for imaging sensors.
- **Memory (RAM):** **1 GB LPDDR2** SDRAM (900 MHz) shared with the GPU ⁶. While 1GB is modest by modern standards, it is sufficient for lightweight data acquisition programs and moderate in-memory processing. The use of LPDDR2 means memory bandwidth is more limited compared to newer boards with DDR3/DDR4, something to consider for data-intensive tasks.
- **Networking: Gigabit Ethernet (RJ45)** via an onboard LAN7515 chip, although throughput is limited to about **300 Mb/s** due to the internal USB 2.0 bus connection ⁷. This is still a significant upgrade from the 100 Mb/s Fast Ethernet on the older 3B (which topped out ~95 Mb/s) ⁸. The 3B+ also has **onboard wireless**: dual-band 2.4 GHz & 5 GHz Wi-Fi (802.11 b/g/n/ac) and Bluetooth 4.2 BLE are integrated via a Cypress/Broadcom combo chip and a PCB antenna ⁹ ⁶. The wireless LAN is dual-band and 802.11ac-capable, providing much better Wi-Fi throughput on 5 GHz compared to the 2.4 GHz 802.11n of the 3B ⁶.
- **USB and I/O:** 4× **USB 2.0** ports for peripherals ¹⁰. The standard **40-pin GPIO header** is present (same pinout as prior models), exposing **multiple I/O interfaces**: e.g. I²C, SPI, UART, PCM/I2S, and GPIO signals for digital sensor interfacing. A **CSI** camera port and **DSI** display port are provided for connecting the official camera module and touchscreen ¹¹. The board also includes a 4-pole 3.5 mm jack providing stereo audio output and composite video output ¹², useful for analog sensors or A/V output (though audio input requires USB sound card or external ADC since no analog input pin is on the Pi). Storage is via a **microSD** slot for the operating system and data storage ¹³ (with the option of USB drives for additional storage).
- **Power:** Requires a **5 V DC supply**, typically via the Micro-USB power port. The recommended input is 5 V at **2.5 A** to ensure sufficient headroom for CPU load and attached peripherals ¹⁴. The 3B+ introduced support for **Power over Ethernet (PoE)** – it includes four PoE header pins that, when combined with an add-on PoE HAT, allow powering the board through the Ethernet cable ¹⁵ ¹⁶.

This is very convenient for clustered deployments, as it reduces separate power wiring when using PoE network switches.

Overall, the Raspberry Pi 3B+ provided a modest but important upgrade over the 3B in CPU speed and connectivity, making it well-suited for networking-intensive applications and slightly heavier processing tasks, all while maintaining full software/hardware compatibility with earlier Raspberry Pi 3 models.

Performance for Data Acquisition, Processing, and Networking

In the context of sensor data acquisition and processing, the Pi 3B+ offers solid performance for a small embedded computer. Its 1.4 GHz quad-core ARM CPU can comfortably handle tasks like reading sensor streams, filtering data, and even moderate real-time processing on multiple threads. For example, early tests indicated roughly a 15% increase in CPU performance over the 3B ³, which, while not dramatic, means tasks complete a bit faster. Computation-heavy jobs (e.g. image processing, machine learning inference) will push the limits of the 3B+; such tasks may need to be distributed across the cluster or use the GPU where possible. The VideoCore IV GPU provides hardware acceleration for graphics and video encoding/decoding (e.g. it can encode 1080p30 H.264 video in real time ⁵), which can be leveraged for camera sensor data to offload work from the CPU.

For data acquisition from typical environmental sensors (temperature, pressure, etc.), the Pi 3B+ has more than enough processing headroom. Reading sensors via I²C or SPI usually consumes minimal CPU time. The Pi's I/O buses support adequate speeds (SPI can operate at tens of MHz, I²C at 100–400 kHz or more in fast mode), allowing high sampling rates. The 1GB RAM is sufficient for buffering sensor data and running data-collection software; however, if performing in-memory analytics on large datasets or images, 1GB can be a limiting factor. In those cases, streaming or incremental processing (or offloading to disk/network storage) is necessary due to memory constraints.

In terms of networking performance, the Pi 3B+'s upgraded network interface is a major benefit for distributed sensor systems. The Gigabit Ethernet (though effectively ~300 Mbit/s max) significantly improves data throughput and latency for node-to-node or node-to-server communication compared to the older 100 Mbit port ⁸. This means the 3B+ can send and receive data three times faster than the 3B under ideal conditions, which is relevant if large volumes of sensor data (for instance, video streams or high-frequency measurements) need to be aggregated or sent to storage. The dual-band Wi-Fi (802.11ac on 5 GHz) can achieve 100+ Mbit/s in practice, providing a decent wireless alternative if wired networking is not feasible, though for a cluster of Pis, wired Ethernet is preferred for stability and consistent throughput.

I/O and data throughput considerations: One limitation of the Raspberry Pi 3 architecture is that the USB ports and the onboard Ethernet controller share the same USB 2.0 bus to the CPU. This means high-speed peripherals (USB disks, cameras, etc.) and network traffic can contend for bandwidth. For example, heavy disk I/O or multiple USB sensors might reduce effective network speed and vice versa. In a data acquisition scenario, it's usually advisable to log data either to the network or to the SD card in a sequential manner to avoid saturating the USB bus. The microSD storage itself can be a bottleneck for data logging; typical SD card speeds (~20–40 MB/s) are much lower than the network bandwidth. If higher disk throughput is needed, a USB 2.0 external SSD can be used (~30 MB/s max over USB2), but again that shares the bus with Ethernet. In summary, the 3B+ can handle moderate data rates from sensors and can network that data effectively, but one must architect around the shared I/O bus bottleneck for optimal performance (e.g. avoid heavy USB transfers while streaming over Ethernet).

Raspberry Pi 3B+ vs. 3B: Thermal Efficiency, Power, and Interfaces

Although the Raspberry Pi 3 Model B will not be used in deployment, it is useful to understand how the 3B+ improved upon its predecessor, as this illustrates the platform's progression:

- **CPU and Thermal Improvements:** The Pi 3B uses the older BCM2837 (Cortex-A53 @1.2GHz) whereas the 3B+ uses the BCM2837B0 stepping, which runs at 1.4GHz and has an integrated heatspreader ³ ⁴. In practical terms, the 3B+ is about 15–17% faster in CPU tasks than the 3B ³. More importantly, thanks to its metal heat spreader and improved power integrity, the 3B+ manages heat better and sustains high clocks longer. Tests have shown that under full CPU load, the 3B+ runs ~10°C cooler than the 3B (66°C vs 75°C in one benchmark) despite using more power, meaning it is less prone to thermal throttling ¹⁷. The 3B often required a heatsink to avoid throttling at 1.2GHz when all cores were busy (it could hit ~80°C under load even with a small heatsink) ¹⁸. By contrast, the 3B+'s stock heatspreader allows it to leverage its higher clock without immediate throttling, as long as ambient temperatures are reasonable ¹⁷. This improved thermal efficiency is a key benefit of the 3B+ in processor-intensive tasks or enclosed deployments.
- **Power Consumption:** The 3B+ does draw more power than the 3B, in line with its performance. Measurements show the 3B+ consuming about 2.3 W at idle (vs ~1.3 W on 3B), and up to ~5.4 W under full load (vs ~3.7 W on 3B) ¹⁹. That's roughly 45% higher power draw at max load for the 3B+ ¹⁷. This higher power usage is the trade-off for the faster CPU and Gigabit PHY. However, because the 3B+ runs cooler, it can use that extra power effectively without throttling, delivering more sustained performance per node ¹⁷. In cluster planning, one should budget about 0.5–1 A (@5V) of current per 3B+ under typical load, and up to ~1.1–1.2 A (6 W) per board in worst case peaks (e.g. heavy CPU + USB activity). The recommended 5V/2.5A power supply provides headroom for attached USB devices and momentary spikes ¹⁴.
- **Networking and Wireless:** The original Pi 3B has 10/100 Ethernet (USB 2.0 LAN9514 chip) and 2.4 GHz 802.11n Wi-Fi (Wireless LAN chip BCM43438). The 3B+ introduced the LAN7515 chip for **Gigabit Ethernet**, boosting wired throughput from ~95 Mb/s to ~300 Mb/s max ⁸. This is a significant upgrade for any network-intensive application or cluster node communications. On wireless, the 3B+ upgraded to the Cypress/Infineon (formerly Broadcom) **CYW43455** chip, which supports dual-band Wi-Fi including 5 GHz 802.11ac and Bluetooth 4.2 ⁶. In contrast, the 3B's Wi-Fi was single-band 2.4 GHz (802.11n) and Bluetooth 4.1. In practice, 3B+ boards can achieve much better Wi-Fi throughput especially in less congested 5 GHz bands – anecdotally, streaming large files or video is more reliable on a 3B+ over Wi-Fi ²⁰. The improved networking makes the 3B+ far more suitable for clustering and data server roles than the 3B.
- **Power-over-Ethernet:** The Pi 3B has no PoE capability on board, whereas the 3B+ includes soldered headers to support PoE via an add-on HAT (accessory). Raspberry Pi released an official PoE HAT (802.3af, ~13 W) that is compatible with the 3B+ ¹⁶. This addition doesn't affect the Pi's function directly, but it greatly simplifies powering multiple units – with 3B+ boards, one can use a PoE network switch and power each Pi through the Ethernet cable, eliminating the need for individual power adapters in a cluster.
- **Other Interfaces:** Both 3B and 3B+ share the same GPIO pinout and peripheral set (HDMI, CSI, DSI, USB, etc.), so from a sensor and accessory standpoint they are identical. One minor change is that

the 3B+ GPIO runs at 3.3 V logic as before, but some pins previously reserved got repurposed for the PoE HAT (the 4-pin PoE header uses GPIO pins that were previously NC). This doesn't affect most usage. Both models have the same 1 GB RAM. Notably, the 3B+'s Wi-Fi module is under a metal shield can, which also helps with EMI and thermals. The form factor (mounting holes and board dimensions) are unchanged, so cases and mounts for Pi 3B generally work for 3B+ as well ²¹.

In summary, the Raspberry Pi 3B+ offers moderate but meaningful improvements: about 15% higher CPU speed, much faster networking, and better thermal/power management. These enhancements make it a better choice for a clustered sensor array system, where network bandwidth and sustained performance are important. The Pi 3 Model B is perfectly capable for many tasks, but its more limited network and thermal headroom could become bottlenecks in the kind of multi-node, data-heavy environment anticipated here. (It's worth noting that some in the single-board computer community have criticized the 3B+'s use of older DDR2 memory and its power design – one Armbian forum user called it “a garbage board” due to “DDR2 with an ok SoC is a bad combination. Bad power delivery, not very usable” ²². While this is a harsh assessment, it underlines the fact that the Pi 3B+ is pushing an aging architecture to its limits, and one should not expect miracles in performance beyond its design.) Overall, the 3B+ strikes a balance of improved capabilities while staying in the \$35 price bracket, making it a pragmatic choice for building a sensor cluster.

Clustering Capabilities and Considerations

Using Raspberry Pi 3B+ boards in a cluster (a multi-node setup) can provide a scalable platform for managing a large array of sensors and distributed processing of their data. However, building a cluster of small boards comes with unique networking, management, and performance considerations. This section discusses how to network and scale Pi 3B+ nodes, tools for cluster management, and performance factors for multi-node systems, including storage and memory constraints.

Networking Setup for Pi Clusters

A robust network is the backbone of any cluster. For the Pi 3B+, **wired Ethernet** is the primary network interface and is strongly preferred for cluster interconnects (the 3B+ has no onboard second NIC – Wi-Fi could be used for external connectivity, but all cluster internal communication typically runs over Ethernet). Each 3B+ has a single 10/100/1000 Ethernet port (300 Mb/s effective). The common approach is to connect all nodes to a central Gigabit Ethernet **switch**. In a simple configuration, one Pi can act as the “head” or master node and others as workers. The head node might also be connected to the wider LAN or Internet (for sending aggregated data or for remote access) via an uplink. For example, an eight-node Pi cluster can be wired to a single switch, with one board designated as the head node that may use an additional USB-Ethernet adapter for external network access, while the built-in Ethernet of all boards is used for the internal cluster network ²³. This separation allows sensor data traffic to stay on the private GigE switch, avoiding congestion with outside network traffic.

Because the Pi 3B+ supports PoE, a popular cluster setup is to use a **PoE-enabled switch**. By equipping each Pi with the PoE HAT, both power and data for each node run over a single Ethernet cable ²⁴. This drastically reduces cable clutter and makes the cluster easier to deploy and maintain (no need for a dozen USB power adapters and cables). Each Pi 3B+ with a PoE HAT can be supplied up to ~13 W (which is plenty, given each uses ~5 W typically) from the switch ¹⁶. If PoE is not used, an alternative is a high-quality multi-port USB power supply (e.g. a 6-port 60 W USB charger) to power multiple Pis, ensuring each port can

supply ~5 V/2.4 A. In any case, careful attention should be paid to the power distribution so that all nodes receive stable 5V under load.

One inherent limitation in Pi clusters is the network throughput and latency relative to traditional PC clusters. Even with the 3B+'s improved Ethernet, 300 Mb/s is modest, and **network bandwidth can become a bottleneck** if the cluster nodes need to exchange large volumes of data frequently. As one Raspberry Pi forum user noted, “the bottleneck is the slow network speed of 100 Mb/s (or 300 Mb/s if you use... the new Raspberry Pi 3 B+)” in Pi clusters ²⁵. This means that distributed workloads which require heavy data shuffling between nodes will not scale well – the time spent communicating can dwarf the time spent computing on these boards. To mitigate this, cluster applications should be designed to minimize inter-node data transfer or to send only compact messages (e.g. summary statistics or filtered sensor readings rather than raw high-rate data streams whenever possible). For tasks that do require significant data sharing, one might consider network topologies or enhancements like using multiple network interfaces (the 3B+ can use a USB 2.0 Ethernet adapter to add a second link, though still limited by USB throughput) or migrating to a platform with true Gigabit/PCIe networking if the bandwidth is mission-critical.

That said, many sensor network tasks are embarrassingly parallel or can be partitioned to avoid saturating the network. For example, if each node reads from a set of sensors and only occasionally sends aggregated data to the master, the 100–300 Mb/s link is likely sufficient. It's also common to dedicate one node as the data aggregator or database server, to which all others send their sensor readings – in this case the inbound traffic concentrates on the head node but remains well below the 300 Mb/s limit if each sensor publishes at a reasonable rate.

Scalability and Cluster Management Tools

The Raspberry Pi 3B+ cluster can scale to a fairly large number of nodes (dozens) in principle, though practical limits will arise from network and power infrastructure, as well as management complexity. Each node runs its own Linux OS instance; there is no shared memory or inherent clustering software in Raspberry Pi OS by default. Thus, to make them work together, one must set up a cluster management or orchestration system.

For **high-performance computing (HPC) style** clusters (e.g. parallel computations), one can use message passing libraries such as **MPI** (MPICH or OpenMPI). Programs must be written or adapted to distribute work across the nodes and communicate via MPI. This is useful for tasks like parallel data processing (e.g. splitting up a computation on sensor data among the nodes). Tools like **SLURM** can be used as a job scheduler on Pi clusters if treating it like a mini-supercomputer. It's important to remember that a cluster of N Raspberry Pis does *not* act like a single N× faster Pi – the overhead of coordination and data exchange is high. As an expert explained, “it's misleading to say that a cluster of 100 Pis has 400 cores and 100 GB of memory” because each Pi's CPU and RAM are separate – one core cannot directly access another board's memory except by sending messages over the network ²⁶. In practice, you still have 100 independent 4-core/1GB computers that need to communicate explicitly. This overhead means linear speedups are rare; clusters only excel for problems that can be parallelized with minimal inter-node communication.

For **orchestration of distributed services or IoT applications**, one might employ containerization or messaging frameworks. Lightweight container orchestration like **Kubernetes (k3s or MicroK8s)** or Docker Swarm can be run on Pi clusters (the 3B+ can run 32-bit or 64-bit Linux containers). This would allow deploying microservices across the Pis – for example, each Pi could run a container that interfaces with its

attached sensors, and a master service collects or processes data. Kubernetes adds a layer of complexity but provides management of the cluster (automatic restarting of services, scheduling, etc.). For smaller clusters or simpler coordination, configuration management tools like **Ansible** (for provisioning and updating all nodes consistently) and custom Python scripts may suffice.

Storage and Memory Constraints in a Pi Cluster

Each Raspberry Pi 3B+ node typically uses a **microSD card** as its boot and storage device. Managing many separate SD cards can be cumbersome, and SD cards have limited write endurance (which might be a concern if each node is logging a lot of sensor data locally). A useful feature of the Pi 3 (and 3B+) is that it supports **network booting** – nodes can boot over the network from a central server, eliminating the need for an SD card per node ²⁷. In a cluster setup, one can configure the head node to act as a DHCP/TFTP server and file server so that the worker Pis boot their OS images from the head node's storage. For instance, one published tutorial describes a cluster where the head node has an external 1 TB SSD, and all seven other Pis netboot from images stored on that SSD; the SSD also provides a shared file storage partition accessible by all nodes ²³. This kind of networked storage approach simplifies maintenance (one place to update the OS or software for all nodes) and improves reliability by removing SD cards (which are a common point of failure). It does, however, consume network bandwidth for disk I/O – effectively shifting disk reads/writes into Ethernet traffic – but on a switched Gigabit network this is usually fine for moderate loads.

If each node does use its own SD card, it's advisable to use high-quality, high-endurance cards and consider using the **ext4 filesystem with journaling disabled** or log management strategies to reduce wear (especially if writing sensor logs frequently). Logging can also be centralized: rather than writing to local disk, sensor nodes could send data to the head node to be written to a database or aggregated log. This again trades network usage for reduced disk stress on the nodes.

In terms of **memory**, each Pi 3B+ has 1 GB RAM, which is not expandable. In a cluster, you cannot simply aggregate the memory (there's no shared memory across nodes), so each task or process on a given Pi is limited to that 1 GB. For many sensor tasks (reading values, simple computations, buffering data), 1 GB is plenty. But tasks like image processing or caching large datasets must be split so that each node handles a portion that fits in its memory. If one needs to work on a larger dataset collectively, an approach is to use distributed computing techniques (split the data into chunks per node) or to use streaming algorithms that process data in pieces. **Storage constraints** also mean that if each node only has, say, a 16 GB SD card, the total storage is distributed – you might not have any single place with lots of free space unless using a network drive. The head node with an external USB HDD/SSD or an NFS server can provide a solution: all nodes can mount a shared network drive where big data (or collected sensor datasets) are stored. This way, the limited local storage isn't a bottleneck, and data is centralized for analysis or offloading to the cloud.

In summary, to mitigate storage/memory constraints in a Pi cluster: use network boot or central storage to reduce reliance on SD cards, carefully plan memory usage per process (possibly use multiple nodes to increase aggregate memory available for an application, albeit with network overhead), and use lightweight data formats to conserve both memory and bandwidth.

Suitability for Sensor Arrays and Interfacing Methods

One of the strengths of the Raspberry Pi platform is its ability to interface with a wide variety of sensors – from simple temperature sensors to cameras – which makes the Pi 3B+ well-suited as a controller in sensor array deployments. The Pi 3B+ provides numerous interface options for connecting sensors and devices:

- **Digital sensor interfaces:** The 40-pin GPIO header exposes **I²C**, **SPI**, and **UART** buses, which are commonly used to connect sensors. For instance, environmental sensors (temperature, humidity, pressure, gas sensors, etc.) often come in I²C or SPI flavors – the Pi can communicate with these directly. The Pi 3B+ has two I²C controllers (one usually dedicated to camera/display internally, one available on GPIO pins by default) and up to 5 UARTs (with one full-featured UART accessible on pins, and a mini UART, depending on configuration). **I²C** is great for multiple small sensors (each with an address on a shared 2-wire bus), whereas **SPI** offers higher throughput for things like ADCs, displays, or high-speed sensors (SPI on Pi can often run at 10+ MHz clock). **UART/Serial** ports can interface with GPS modules, serial sensors, or even microcontroller-based sensor nodes. Additionally, one GPIO pin can be configured for **1-Wire** protocol to read simple devices like DS18B20 temperature sensors. The Pi's flexibility with GPIO means custom digital protocols can be bit-banged if needed (at some CPU cost).
- **Analog sensors:** The Pi notably lacks built-in analog input pins (unlike microcontroller boards). To use analog-output sensors (e.g. certain light, pH, or analog temperature sensors), one typically adds an **ADC chip** (analog-to-digital converter) via SPI or I²C. There are many ADC add-on boards (e.g. MCP3008 8-channel 10-bit ADC over SPI) that are compatible. These allow the Pi to read analog voltages from sensors. Alternatively, one could use an Arduino or other microcontroller to read analog sensors and then send the readings to the Pi via serial or I²C.
- **Imaging and video sensors:** The Pi 3B+ has a **CSI camera port** which supports the official Raspberry Pi Camera Modules (e.g. the 8 MP and 12 MP camera units) and some third-party camera sensors. This is ideal for building camera arrays or machine vision setups – each Pi can attach a camera and capture images or video. The VideoCore IV GPU can handle the camera data efficiently and even encode video on the fly. For multiple cameras beyond one per Pi, one could use USB cameras, but on a single Pi 3B+ the USB bandwidth is limited (as discussed). In a cluster, however, you could attach one camera per node and effectively have a multi-camera distributed system (each node processing its own camera feed). For example, a synchronized camera array could be created with a cluster of Pis, where each Pi+camera captures from a slightly different perspective. (There are advanced setups like the ArduCAM synchronization HAT that allow multiple cameras on one Pi, but those are special cases.)
- **Other sensors/actuators:** The Pi's GPIO can also generate PWM signals (for controlling servos, motors, or dimming LEDs) and capture inputs like triggers or pulses from devices. This means a Pi cluster can not only read sensors but also control actuators in the environment (e.g. controlling a sensor positioning system, or activating relays). The 3B+ has the same capabilities here as the 3B – for instance, hardware PWM channels on GPIO for precise control of duty cycles.

Given this rich interfacing capability, a cluster of Pi 3B+ boards could manage a large sensor array by assigning groups of sensors to different nodes. For example, in an environmental monitoring scenario, one Pi might handle a suite of weather sensors (temp, humidity, pressure via I²C), another Pi might interface

with a high-resolution camera or a software-defined radio, and another might collect data from a network of distributed microcontroller-based sensors. They can then synchronize or aggregate their readings over the network.

Synchronization and Data Coordination Across Nodes

When multiple nodes are each collecting data, an important consideration is **time synchronization** and coordination so that sensor readings can be aligned or fused meaningfully. The Raspberry Pi 3B+ does not have built-in precision timing beyond the system clock, so the typical approach is to use **NTP (Network Time Protocol)** to keep all nodes' clocks in sync. One Pi (often the head node) can act as an NTP server (master), while the others run NTP clients to sync to it ²⁸. In practice, NTP can usually keep Raspberry Pi clocks within a few milliseconds of each other on a local network – often sufficient for environmental sensors or even aligning video frames within a few ms. If tighter synchronization is required (sub-millisecond), one might consider **PTP (Precision Time Protocol)** or a custom synchronization signal. For example, some multi-camera arrays use a GPIO as a trigger line: one master Pi toggles a GPIO that is electrically connected to GPIO inputs on other Pis to trigger camera capture simultaneously. This kind of hardware sync can achieve microsecond-level alignment, but requires extra wiring. Over Ethernet, approaches like sending a multicast “sync now” message have some variability due to network stack latency, but for many use cases, a well-configured NTP is enough.

Another coordination strategy is at the application level using timestamps. Each node can timestamp its sensor readings (using its local clock) and send the data to a central node or database. As long as the clocks are roughly in sync, the data can be merged or compared by timestamp. If absolute synchronization to real-world time is needed (e.g. for scientific data collection), one can also equip one node with a GPS module to act as a Stratum 1 time source and discipline the cluster clocks that way (GPS provides an accurate time reference).

In terms of orchestrating data collection across nodes, lightweight messaging and signaling protocols are often used. For instance, a **publish/subscribe model** with a broker can coordinate readings: one node (or an external controller) can publish a “sync” or “start sampling” message to a topic that all sensor nodes subscribe to, prompting them to take a measurement at the same moment. They then publish their sensor data to their own topics. This can be done with MQTT or similar. The latency over Ethernet for a small message is low (a few hundred microseconds to a few milliseconds), so for many sensors this is effectively simultaneous.

If using the cluster for more computational tasks (e.g. distributed processing of sensor data), coordination might involve using libraries or frameworks that handle task distribution. For example, in a multi-node image processing scenario, one could use OpenCV on each node to handle its camera feed, and use ZeroMQ or MPI to aggregate results.

Software Environment and Messaging in the Cluster

The Raspberry Pi 3B+ can run a variety of Linux-based operating systems. For a clustered sensor array project, a common choice would be a customized **Raspbian (Raspberry Pi OS)** or an upstream Debian-based distro. Raspberry Pi OS (formerly called Raspbian) is essentially Debian optimized for the Pi's hardware. As of 2025, Raspberry Pi OS has 32-bit and 64-bit versions (the 3B+ being 64-bit capable). One could also opt for a plain Debian install (e.g. Debian 12 “Bookworm” or the upcoming Debian 13 “Trixie”) on

the Pi – these are supported thanks to the Raspberry Pi bootloader and kernel being able to work with mainline Debian. In fact, “Trixie” is just the codename for Debian testing (future Debian 13) ²⁹, and a Pi 3B+ can run it, although it may require some tweaks for firmware. An alternative community OS is **Armbian**, which historically provided optimized Debian/Ubuntu images for various ARM boards. (Armbian had experimental builds for Raspberry Pi, but official support was dropped in recent years, as the Raspberry Pi Foundation’s own OS covers the Pi well ³⁰.)

For deployment, a **lightweight headless OS** is desirable – e.g. Raspberry Pi OS Lite (console-only) – to minimize unnecessary services and GUI overhead. This leaves more CPU and memory for sensor and network tasks. The OS can be tailored with needed kernel modules (enable I²C, SPI, etc. via `raspi-config` or device tree overlays) and packages.

Each node in the cluster will likely run a suite of software including: sensor interface libraries, data processing scripts, and communication services. In Python (a common language on Pi), one might use libraries such as **RPi.GPIO** or **gpiozero** for GPIO sensor control, **smbus2** for I²C communications, and specific sensor libraries from the manufacturer or community (for example, Adafruit’s CircuitPython libraries support many I²C/SPI sensors on Raspberry Pi). If using the camera, the official `libcamera` stack or OpenCV can capture images.

For the cluster communication and orchestration, there are a few approaches, often used in combination:

- **Messaging Protocols (MQTT, ZeroMQ, etc.):** A publish/subscribe broker like **MQTT** is a natural fit for sensor networks. For instance, each Pi can run an MQTT client that publishes its sensor readings to specific topics (e.g. `node3/temperature`) and subscribes to control topics (e.g. `allnodes/sync`). An MQTT broker can run on the head node (or another dedicated node). In fact, the simplest way to link a web service and Raspberry Pi sensors is often to set up an MQTT broker as an IoT hub ³¹. The open-source **Mosquitto** broker runs well on the Pi and can facilitate this communication. Using MQTT decouples the data producers (sensor nodes) from consumers (could be a central logger, or user interface, etc.), and it’s lightweight enough for the Pi’s resources. Alternatively, a **ZeroMQ** distributed messaging pattern can be used: ZeroMQ allows direct socket-based messaging in various patterns (pipeline, publish-subscribe, request-reply) without a central broker. Some Pi cluster builders have had success with ZeroMQ for sending messages between Pis, finding it a “great solution” for coordinating distributed processes ³². ZeroMQ can be embedded into Python or C++ programs for custom messaging (for example, to have nodes send data to a master process reliably).
- **Custom Brokers or RPC:** One could implement a simple custom server on the head node (for example, a Python Flask or FastAPI service or a custom TCP server) to collect data from nodes. Nodes would then just HTTP POST or send TCP/UDP packets with sensor data. This might be sufficient for small clusters and straightforward to implement, but it can become harder to manage as the system scales. Nonetheless, a custom broker can be tuned exactly to the project’s needs (e.g. a specialized binary protocol for sensor data).
- **Orchestration and Monitoring:** It’s important to deploy code and configurations to all nodes consistently. Tools like **Ansible** can massively simplify this by running the same setup steps on all Pis via SSH. For runtime monitoring, one might use something like **Prometheus + Grafana** to gather metrics from each node (CPU usage, temperature, network) – this is particularly useful in a cluster to ensure none of the nodes are overloaded or overheating. There are also cluster-specific monitoring

tools (Kubernetes has a whole ecosystem, but even without k8s, one could run Node-Exporter on each Pi and aggregate stats). Given that the Pis will be deployed possibly in the field or lab for sensor duties, maintaining insight into their health (and being able to update software remotely) is crucial.

- **Example software stack:** Suppose we deploy Debian Trixie on each Pi 3B+ with Python3 environment. We could install Paho MQTT library (for MQTT client), a sensor library for each sensor type, and perhaps an MPI library if doing any parallel computing tests. The head node might run a TimescaleDB or InfluxDB time-series database to store all sensor readings and an MQTT broker (Mosquitto). The worker nodes on boot connect to the broker and start publishing their sensor data periodically. If a more command-and-control setup is needed, the head node could send out a message like “take sample now” to all nodes, or even use a tool like **cron** or **schedule** to have nodes measure at synchronized intervals. The messaging layer ensures loose coupling – for example, if one node goes down, others are unaffected and the broker can last-will message or log the outage.

All of this software runs atop Linux – meaning standard Linux networking and security practices apply (SSH for access, possibly VPN if deployed remotely, etc.). The Pi 3B+ handles modern Linux fine, though one should avoid overly heavy software (a full desktop environment or Java application server on each might be too slow). However, running a Java-based application (like some IoT frameworks) is not out of the question – e.g. there are cases where a Pi cluster was running a JVM on each node; the 1GB RAM is the limiting factor in those scenarios.

In summary, the software environment for a Pi 3B+ cluster will likely be a pared-down Linux with just the necessary services. The use of messaging protocols like **MQTT** can greatly simplify data collection from multiple nodes by providing a reliable, asynchronous channel for sensor data ³¹. More advanced setups might involve containerization or even a mini Kubernetes cluster (which the Pi 3B+ can handle, although memory is tight, so typically Pi 4s are used for k8s). The exact choices depend on whether the emphasis is on ease of use, reliability, or performance.

Power, Cooling, and Enclosure Considerations for Cluster Deployment

When deploying a cluster of Raspberry Pi 3B+ boards, especially for sensor arrays in field or lab conditions, careful thought must be given to power delivery, cooling, and physical enclosures:

- **Power Distribution:** As mentioned, each Pi 3B+ ideally has a 5 V/2.5 A power source ¹⁴. In a clustered setup, this can be managed via PoE (if available) or using multi-port USB power supplies. For example, an 8-node cluster could be powered by two 4-port 5V/10A power banks, or a single industrial power supply feeding multiple step-down converters to individual Micro-USB cables. The goal is to ensure **voltage stability**; long USB cables or thin wires can cause voltage drop, especially under load, leading to the Pi’s low-voltage warning. Clusters in the field might even use battery or solar setups – in such a case, a DC/DC regulator would provide 5 V to each Pi from the battery pack. Keep in mind surge currents: the Pi can draw more current on boot or when the CPU ramps up, so leave headroom in the power budget. Using PoE is often the cleanest solution: a single Ethernet cable per node from a PoE switch, as demonstrated in an official 8-Pi cluster example (with PoE+ HATs, no separate power cables needed) ²⁴.

- **Cooling Requirements:** The Pi 3B+ runs cooler than the 3B under load due to its heat spreader, but it *can* still run warm, especially if enclosed with other boards. In a multi-node cluster, if boards are stacked close together, heat can build up. For lab clusters, one often sees stacks of Pis with small **heatsinks** on the SoC and perhaps a couple of **5 V fans** moving air through the stack. For example, a cluster case might include a fan blowing horizontally across all boards, or each layer of the stack may be spaced with standoffs to allow airflow. If the cluster is deployed in a field enclosure (like a weather-proof box for environmental sensing), thermal management is even more important: high ambient temperatures and sun exposure can push the Pi's temperature up. In such scenarios, active cooling (fans) or at least passive measures (heat sinks, ventilation holes, sunshade) should be implemented. The 3B+ will throttle at 85 °C to protect itself ³³ – it's better to prevent reaching that. Empirically, a Pi 3B+ with no heatsink in open air might reach ~60–70 °C under continuous load ³⁴, which is fine. But inside a box with multiple Pis, temperatures could go higher. Consider using temperature triggers (the Pi's firmware can toggle a GPIO to control a fan at a set temperature, or a simple script can do so via the `/sys` thermal readings).
- **Enclosure Design:** For a cluster of multiple boards, there are off-the-shelf **rack or case solutions** (e.g. cluster cases that hold 4 or 8 Pi boards with spacing) or you can custom-build using acrylic layers or 3D-printed frames. In a lab setting, a popular option is to mount Pis on a rack with spacers – perhaps even a 1U rack shelf can hold several Pi boards – and use the lab's network rack for switching and power. In the field, if the cluster needs to be co-located (say, a distributed sensor system all within one outdoor box), you'll need an enclosure that protects from weather and dust (IP-rated boxes) and also manages heat. One approach is to use a large junction box with a heat sink plate; attach the Pi boards to the inside of the plate so their heat spreads to the outside. Another is to have a weatherproof box but include an intake/exhaust with filtered fans (maybe necessary if the cluster dissipates $\sim 5\text{ W} \times N$; for 8 nodes that's $\sim 40\text{ W}$, which will definitely heat up a sealed box). **Cable management** is also a consideration – using PoE greatly reduces the number of cables (just one per node). If not using PoE, you'll have both Ethernet and power cables for each Pi; ensure the enclosure or rack can accommodate that and strain-relieve the connections.
- **Field deployment power:** If this sensor array cluster is used in remote or outdoor locations, power might come from solar panels and batteries. The Pi 3B+ is not the lowest power device, but you can implement power-saving (e.g. turn off HDMI, unused USB, maybe use the `raspi-config` to disable Wi-Fi/BT if not needed to save a bit of power). In a cluster, perhaps not all nodes need to run at full power continuously – it's possible to duty-cycle some nodes if the sensor application allows (for example, have half the nodes sleep or idle if measuring infrequently). Also consider using the Pi's boot-up behavior: you could have the cluster nodes automatically boot when power is applied in the morning (when solar is available) and safely shutdown when batteries get low, etc., using a supervisory microcontroller or a UPS HAT on the head node.

In summary, **reliability** in power and cooling is key to a successful cluster deployment. Fortunately, the Raspberry Pi community has ample experience with multi-Pi setups, and many best practices have emerged: use quality PSUs or PoE, add heatsinks/fans in multi-board enclosures, and avoid enclosing them in tight spaces without airflow. By following these practices, one can keep a Pi 3B+ cluster running 24/7 in both lab and moderate outdoor conditions. (The official operating temperature range of the Pi 3B+ is 0–50 °C ³⁵, so if your environment goes beyond that, extra cooling or heating measures might be needed – for instance, in sub-zero conditions, you may need to keep the box slightly warm to avoid issues with components like the camera or sensors.)

Limitations and Mitigation Strategies

Finally, it's important to acknowledge the limitations of a Raspberry Pi 3B+ cluster and discuss strategies to mitigate potential bottlenecks in bandwidth, CPU, and thermal performance:

- **Network Bandwidth Bottleneck:** As discussed, the cluster's internal network (Ethernet) tops out around 300 Mb/s per node. This is modest compared to PC or server networks, and can be a bottleneck if the application requires streaming large data between nodes. Mitigations include: designing the system so that **most processing is done locally** on each node (send results, not raw data), using efficient data encoding/compression for any inter-node communication, and if applicable, structuring communication in a hierarchical way (e.g. nodes send data to a master in a tree pattern rather than all-to-all flooding). In some cases, adding an extra network interface can help segment traffic (for instance, a second USB Ethernet for out-of-band management or internet access, while the on-board Ethernet is dedicated to inter-node traffic). However, remember all NICs on a Pi 3B+ share the same USB 2.0 bus, so the total throughput is still limited. If extremely high bandwidth is needed, one might consider upgrading to a different platform (like Raspberry Pi 4 which has true Gigabit + USB3) for the master node or for certain roles. In summary, **minimize network chatter** – only send what you need to – and your Pi cluster will perform much better.
- **CPU and Processing Limitations:** Each Pi 3B+ has a relatively modest CPU by today's standards – four Cortex-A53 cores that, while 64-bit, are in-order cores with performance roughly comparable to an entry-level laptop from many years ago. Certain tasks like heavy image processing, crypto, or large database queries can easily saturate the CPU. In a cluster, adding more nodes can linearly increase aggregate CPU throughput *only if* the problem can be parallelized effectively. Tasks that are single-threaded or require tight coupling won't speed up much on more Pi nodes. To mitigate CPU limitations, you can: offload work to specialized hardware when possible (e.g. use the GPU for video encoding or use accelerators), ensure your code is optimized (make use of the NEON SIMD instructions on the Pi for math, or use C/C++ extensions for Python in critical loops), and choose algorithms that trade off work between nodes efficiently. If a particular node is overburdened (say one Pi is both collecting camera data and performing analysis on it), consider splitting roles – e.g. one Pi captures images and sends them to another Pi dedicated to processing. By pipeline-ing tasks across nodes, you can use the cluster to overcome the CPU limits of any single node. Another strategy is simply to accept slower processing and schedule tasks appropriately (not everything needs to be real-time; batch processing of logged data during idle periods can work if real-time response isn't critical). Keep in mind also that with only 1 GB RAM, swapping on the Pi (to SD card) is extremely slow – avoid memory exhaustion which causes swapping, as it will grind the CPU to a halt. If you have memory-heavy tasks, try to partition them or use streaming approaches that work in chunks.
- **Thermal Throttling and Heat Dissipation:** While the 3B+ improved thermal behavior, a cluster of them can still run into heat issues, especially in a confined space or warm environment. If a Pi's CPU reaches 85 °C, it will throttle down its frequency, drastically reducing performance. The mitigation strategy is straightforward: **improve cooling**. This might mean adding small adhesive heatsinks on the SoC, using active cooling (a fan blowing through the cluster or individual mini-fans on each Pi), and spacing out the boards. Empirical data suggests the 3B+ can sustain its 1.4 GHz speed under load as long as it stays under throttle temperature ¹⁷. A roughly 10 °C temperature reduction was observed in the 3B+ vs 3B under load due to the heat spreader ¹⁷, but if you stack many 3B+

together without airflow, they could still climb towards throttling. In a lab, simply ensuring some air circulation (even a desk fan pointed at the cluster) can be an effective mitigation. In enclosed deployments, consider heat sinks and perhaps a thermostatically controlled fan. The power consumption (and thus heat generation) can also be reduced by capping the max CPU frequency if needed – for example, you could configure the Pi to max out at 1.2 GHz (like the old 3B) if you find heat to be a problem, though this sacrifices performance. Another mitigation is leveraging the cluster: if one node is running too hot, redistribute some tasks to other nodes if they have capacity, so no single board is over-stressed.

- **Reliability and Failover:** An often under-appreciated limitation of using many SD-card based systems is the risk of individual node failures (SD cards can corrupt, nodes can crash). While not a performance bottleneck per se, it's part of limitations. The strategy here is to design the system with **redundancy** and easy reconfiguration. For instance, keep backups of each node's OS image, or use network boot so nodes are diskless and easier to replace. If a node fails, the system should ideally detect it (the head node can monitor heartbeats from each Pi) and possibly alert an operator or shift the workload to other nodes if possible. In a sensor network, losing one node might mean losing some sensors – unless you have overlapping coverage or redundancy in sensor placement. Planning for this at the architecture level (maybe duplicate critical sensors on two nodes, etc.) can mitigate the impact of a single board failure.

To conclude, a Raspberry Pi 3B+ cluster is a powerful tool for managing sensor arrays, offering a balance of computing, connectivity, and flexibility. However, one must work within its constraints: **limited bandwidth, modest CPU, and power/thermal limits**. By using the strategies outlined – efficient data handling to avoid network floods, parallelizing tasks to use multiple nodes, cooling the hardware, and using appropriate software protocols – you can mitigate these bottlenecks. Clusters of Pis have been successfully used in academic and hobbyist projects as miniature distributed systems. For example, researchers have built 4-node Pi 3B+ clusters to tackle parallel computations like matrix multiplication, finding that they form a coherent, energy-efficient cluster for enhanced processing via distributed computing methods ³⁶. While they obviously cannot match the raw performance of large servers, they excel in **cost-efficiency and scalability** for IoT and sensor network applications. With careful design, a Pi 3B+ cluster can effectively manage a complex sensor array, providing both the real-time interfacing capabilities and the distributed processing needed for collecting, processing, and transmitting sensor data in a robust manner.

Sources:

1. Raspberry Pi 3 Model B+ Product Brief, Raspberry Pi (2018) ³⁷ ³⁸
2. Seeed Studio Blog – *Raspberry Pi 3 B+ Comparison with 3B* (2019) ³⁹ ⁴⁰
3. Raspberry Pi Documentation – BCM2837B0 Processor Notes ⁴
4. Datenreise.de – *Raspberry Pi 3B+ and 3B in comparison* (2018) ⁴¹ ¹⁷
5. Raspberry Pi Forums – discussion on Pi cluster networking (2018) ⁴² ²⁶
6. Armbian Community Forums – user feedback on Pi 3B+ (2019) ²²
7. Raspberry Pi Forums – on synchronizing Pi clocks via NTP ²⁸
8. Raspberry Pi Forums – on using ZeroMQ between Pis (2017) ³²
9. Raspberry Pi Forums – on using MQTT for Pi sensor data (2020) ³¹
10. Research Paper – Dubey & Kagdi, *Raspberry Pi Cluster for parallel computing* (2020) ³⁶

1 2 5 7 9 10 11 12 13 14 15 21 35 37 38 datasheets.raspberrypi.com

<https://datasheets.raspberrypi.com/rpi3/raspberry-pi-3-b-plus-product-brief.pdf>

3 8 16 17 19 33 34 41 Raspberry Pi 3B+ and 3B in comparison

<https://www.datenreise.de/en/raspberry-pi-3b-and-3b-in-comparison/>

4 1 | PDF

<https://www.scribd.com/document/634676158/1>

6 39 40 Raspberry Pi 3 B+ Comparison with 3B and More! - Latest News from Seeed Studio

<https://www.seeedstudio.com/blog/2019/11/29/raspberry-pi-3-b-comparison-with-3b-and-more/?srsltid=AfmBOoopkKhdcS91iibO8UEjr9U9pbJEr7xbeYxY4ag6rzMYArVNDQC>

18 SBC consumption/performance comparisons - Reviews, Tutorials, Hardware hacks - Armbian Community Forums

<https://forum.armbian.com/topic/1748-sbc-consumptionperformance-comparisons/>

20 Raspberry Pi 3 vs Raspberry Pi 3 B+ : r/RetroPie - Reddit

https://www.reddit.com/r/RetroPie/comments/9hfrnf/raspberry_pi_3_vs_raspberry_pi_3_b/

22 List of Stuff - Off-topic - Armbian Community Forums

<https://forum.armbian.com/topic/10971-list-of-stuff/>

23 24 Build your Raspberry Pi cluster and unlock distributed computing

<https://blackdevice.com/build-raspberry-pi-cluster-unlock-distributed-computing/>

25 26 27 42 raspbian - Raspberry Pi 3 Cluster - Raspberry Pi Stack Exchange

<https://raspberrypi.stackexchange.com/questions/82321/raspberry-pi-3-cluster>

28 Best way to synchronize two Raspberry Pi3

<https://forums.raspberrypi.com/viewtopic.php?t=151605>

29 DebianTrixie - Debian Wiki

<https://wiki.debian.org/DebianTrixie>

30 Why is Raspberry Pi OS based on debian?

<https://forums.raspberrypi.com/viewtopic.php?t=285315>

31 Publishing sensor information from Raspberry Pi 4 using MQTT

<https://forums.raspberrypi.com/viewtopic.php?t=285879>

32 ZeroMQ help - many to one - Raspberry Pi Forums

<https://forums.raspberrypi.com/viewtopic.php?t=188817>

36 (PDF) Run Time Analysis of Matrix Multiplication Using Raspberry Pi Cluster Supercomputer

https://www.researchgate.net/publication/341271587_Run_Time_Analysis_of_Matrix_Multiplication_Using_Raspberry_Pi_Cluster_Supercomputer